Lab Outcomes:

- Understand how to use C# strings for various operations like concatenation, comparison, substring extraction, search, trimming, and replacement.
- Implement string interpolation for easier and more readable string concatenation.
- Use loops (for, while, foreach) to iterate through code and process multiple elements.

C# Strings

In C#, string is an object of **System.String** class that represent sequence of characters. We can perform many operations on strings such as concatenation, comparison, getting substring, search, trim, replacement etc.

string vs String

In C#, *string* is keyword which is an alias for *System.String* class. That is why string and String are equivalent. We are free to use any naming convention.

```
string s1 = "hello";//creating string using string keyword
String s2 = "welcome";//creating string using String class
```

C# String Concat() Method

```
string firstName = "John ";
string lastName = "Doe";
string name = firstName + lastName;
Console.WriteLine(name);
```

```
string s1 = "Hello ";
string s2 = "C#";
Console.WriteLine(string.Concat(s1,s2));
```

String Interpolation

String interpolation is a better way of concatenating strings. We use + sign to concatenate string variables with static strings.

C# 6 includes a special character $ to identify an interpolated string. An interpolated string is a mixture of static string and string variable where string variables should be in {} brackets.

```
string firstName = "John";
string lastName = "Doe";
string name = $"My full name is: {firstName} {lastName}";
Console.WriteLine(name);
```

C# String ToLower()

```
string s1 = "Hello C#";
string s2 = s1.ToLower();
Console.WriteLine(s2);
```

C# String ToUpper()

```
string s1 = "Hello C#";
string s3 = s1.ToUpper();
Console.WriteLine(s3);
```

C# String IndexOf()

Return the index/location of character.

```
string s1 = "Hello C#";
int index = s1.IndexOf('e');
Console.WriteLine(index);
```

C# Conditional Statements

In C# programming, the *if statement* is used to test the condition. There are various types of if statements in C#.

- if statement
- if-else statement
- nested if statement

if statement

```
int num = 10;
if (num % 2 == 0)
{
   Console.WriteLine("It is even number");
}
```

If else statement

```
   int num = 11;
   if (num % 2 == 0)
   {
      Console.WriteLine("It is even number");
   }
   else
   {
      Console.WriteLine("It is odd number");
   }
```

C# switch

The C# *switch statement* executes one statement from multiple conditions. It is like if-else-if ladder statement in C#.

```
Console.WriteLine("Enter a number:");
int num = Convert.ToInt32(Console.ReadLine());

switch (num)
{
   case 10: Console.WriteLine("It is 10"); break;
   case 20: Console.WriteLine("It is 20"); break;
   case 30: Console.WriteLine("It is 30"); break;
   default: Console.WriteLine("Not 10, 20 or 30"); break;
```

C# For Loop

The C# *for loop* is used to iterate a part of the program several times. If the number of iterations is fixed, it is recommended to use for loop than while or do-while loops.

The C# for loop is same as C/C++. We can initialize variable, check condition and increment/decrement value.

```
for(int i=1;i<=10;i++){
  Console.WriteLine(i);
}
```

C# While Loop

In C#, *while loop* is used to iterate a part of the program several times. If the number of iterations is not fixed, it is recommended to use while loop than for loop.

```
int i=1;
while(i<=10)
{
    Console.WriteLine(i);
    i++;
}
```

C# Foreach Loop

The C# *foreach loop* is used exclusively to loop through elements in an array.

```
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
foreach (string i in cars)
{
 Console.WriteLine(i);
}
```

Lab Tasks:

Q1. Create a basic calculator in C# Console. Input must be taken from user for every calculation.

Q2. Write a C# program that asks the user to enter their full name and then displays the name in uppercase, lowercase, and also shows the total number of characters in it using string methods. Replace the space between first and last name with underscore (_).

Q3. Use Math functions to perform advance functions of calculator i.e., rounding off a value, calculating square root finding out max or min value from multiple use inputs. To store values for constant variables, use constants of C# i.e., pi value etc.

Q4. Create a C# application where the user enters any integer, and the program determines whether the number is an Armstrong number or not. An Armstrong number is one whose digits raised to the power of the number of digits add up to the original number (e.g., $153 \rightarrow 1^3 + 5^3 + 3^3 = 153$). Use loops to extract digits, if-else to compare results, and display a final message stating whether the number qualifies as an Armstrong number.

Lab Assessment Criteria:
Lab #03 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 03
Rubric Evaluation Guideline:

#	**Qualities & Criteria**	**0 < Poor <=1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER1	Task/Program Completion	Minimal or no task functionality was achieved	Some tasks were completed but few has errors or incomplete functionalities.	All tasks were completed without any errors.
#	**Qualities & Criteria**	**0 < Poor <=1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	Code Structure and Design	Poor code structure and missing essential code elements like functions, data structures, or control flow.	Code functions but lacks proper structure, documentation, or uses inefficient algorithms.	Well-structured, documented code using appropriate data structures and algorithms.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1< Satisfactory <= 2.5**	**2.5< Excellent <= 4**
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.